

Componentes del lenguaje JavaScript

-1. Introducción a JavaScript

¿Cuál es la historia de JavaScript?

JavaScript es un lenguaje de programación que se utiliza para agregar interactividad a las páginas web. Fue creado por Brendan Eich en 1995, mientras trabajaba en Netscape Communications Corporation. Fue originalmente llamado Mocha, luego renombrado a LiveScript, y finalmente se convirtió en JavaScript.

JavaScript fue diseñado originalmente para ser un lenguaje de scripting(diseñado para escribir scripts) para el navegador Netscape Navigator. En ese momento, las páginas web eran principalmente estáticas y no ofrecían muchas formas de interactuar con el usuario. JavaScript permitió a los desarrolladores web crear páginas más dinámicas e interactivas.

JavaScript ganó popularidad rápidamente, ya que permitió a los desarrolladores crear efectos visuales en tiempo real y responder a la entrada del usuario sin necesidad de recargar la página. Los desarrolladores también comenzaron a utilizar JavaScript para validar los formularios de entrada de datos y para crear juegos simples.

En 2002, la especificación de JavaScript fue estandarizada por la Organización Internacional de Normalización (ISO) y la Comisión Electrotécnica Internacional (IEC) como ECMA-262. Esta especificación ha sido revisada varias veces desde entonces, y la última versión en el momento de escribir esto es la ECMA-262, edición 11, lanzada en 2020.

La especificación de JavaScript es mantenida por la organización sin fines de lucro ECMA International, y ha pasado por varias revisiones importantes. Siendo la versión más reciente ECMAScript 2022, también conocida como ES2022. JavaScript es ahora uno de los lenguajes de programación más populares y ampliamente utilizados en todo el mundo, y sigue evolucionando significativamente y ahora se puede utilizar para crear aplicaciones de una sola página (SPA), aplicaciones de servidor con Node.js, aplicaciones de escritorio con Electron y mucho más.

¿Cuáles son los usos principales de JavaScript?

JavaScript es un lenguaje de programación que se utiliza principalmente en el desarrollo web para agregar interactividad y dinamismo en las páginas web. Pero no se queda solo en eso, también se puede utilizar en el lado del servidor con Node.js para desarrollar aplicaciones web y servicios web. Esto permite que los desarrolladores utilicen el mismo lenguaje en el lado del cliente y del servidor, lo que puede simplificar la arquitectura de una aplicación

Algunos de los usos más comunes de JavaScript son :

1. **Validación de formularios:** JavaScript se utiliza a menudo para validar los datos que se ingresan en los formularios web antes de enviarlos al servidor. Esto ayuda a garantizar que los usuarios proporcionen información válida y completa en los campos requeridos.
2. **Efectos visuales y animaciones:** JavaScript también se utiliza para crear efectos visuales y animaciones en las páginas web. Esto puede incluir efectos de desplazamiento, transiciones, animaciones de carga y mucho más.
3. **Manipulación del DOM:** JavaScript se utiliza a menudo para manipular el Document Object Model (DOM) de una página web. El DOM es la representación del árbol de objetos de una página web y JavaScript puede utilizarse para agregar, eliminar o modificar elementos en el DOM.
4. **Comunicación con el servidor:** JavaScript se utiliza a menudo para comunicarse con el servidor y actualizar los datos de la página web sin necesidad de recargarla. Esto se puede lograr utilizando técnicas como AJAX (Asynchronous JavaScript and XML).

¿Cómo integrar JavaScript en un documento html?

1. **Usando la etiqueta script:** Se tiene que agregar la etiqueta script dentro de la sección head o body del documento HTML y dentro de ella escribir el código JavaScript. Por ejemplo:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Mi página web</title>
  </head>
  <body>
    <h1>Bienvenidos a mi página web</h1>

    <script>
      // Aca escribir el código JavaScript
    </script>
  </body>
</html>
```

2. **Agregando un archivo externo:** Se puede crear un archivo con extensión .js y agregarlo al documento HTML usando la etiqueta script y especificando el atributo src con la ruta del archivo. Por ejemplo:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Mi página web</title>
  </head>
  <body>
    <h1>Bienvenidos a mi página web</h1>

    <script src="ruta/a/mi/archivo.js"></script>
  </body>
</html>
```

¿Cómo los navegadores interpretan javascript dentro de un documento html?

Cuando un navegador encuentra una etiqueta script en un documento HTML, interpreta el contenido de esa etiqueta como código JavaScript y lo ejecuta en el contexto de la página web. La forma en que se interpreta y ejecuta el código JavaScript depende del motor de JavaScript del navegador.

El proceso de interpretación y ejecución de JavaScript consta de los siguientes pasos:.

1. **Análisis léxico:** el código JavaScript se divide en tokens, que son unidades de significado, como palabras clave, identificadores, números, etc.
2. **Análisis sintáctico:** los tokens se combinan en una estructura de árbol de sintaxis abstracta (AST) que representa la estructura gramatical del código.
3. **Análisis semántico:** se verifica que el código JavaScript tenga un significado válido y se resuelven los identificadores (variables, funciones, etc.) en sus valores correspondientes.
4. **Ejecución:** el código JavaScript se ejecuta en el contexto de la página web, lo que significa que tiene acceso a los elementos HTML y al DOM (Document Object Model) de la página. Durante la ejecución, se realizan operaciones como la asignación de valores a variables, la llamada a funciones, la manipulación del DOM, etc.

En resumen, los navegadores interpretan el código JavaScript dentro de un documento HTML utilizando un proceso de análisis y ejecución que se realiza en tiempo real mientras la página se carga y se muestra al usuario.

-2. Fundamentos de JavaScript

¿Cuáles son los tipos de datos en JavaScript?

En JavaScript, existen varios tipos de datos que se utilizan para almacenar diferentes tipos de valores. Estos son de tipos de datos primitivos y complejos.

Datos primitivos:

1. **number:** el tipo de dato número en JavaScript incluye todos los números, enteros o de punto flotante. Esto significa que puede almacenar tanto enteros como decimales. Se pueden realizar operaciones matemáticas con números en JavaScript, como suma, resta, multiplicación y división.
2. **string:** el tipo de dato cadena en JavaScript se utiliza para almacenar cadenas de texto. Pueden estar compuestas por una o varias palabras, números, caracteres especiales y espacios. Las cadenas se pueden concatenar para crear una cadena más larga.
3. **boolean:** el tipo de dato booleano en JavaScript incluye dos valores: true (verdadero) y false (falso). Se utiliza para representar el valor de verdad de una expresión, por ejemplo, si una condición es verdadera o falsa.
4. **null:** el valor null se utiliza para representar la ausencia de valor. Es decir, si una variable tiene asignado el valor null, significa que no tiene ningún valor.
5. **undefined:** el valor undefined se utiliza para representar una variable que no ha sido asignada o a la que no se ha dado ningún valor. En otras palabras, si una variable se ha declarado pero no se ha inicializado con ningún valor, su valor es undefined.
6. **symbol:** el tipo de dato symbol se utiliza para crear identificadores únicos para objetos. Cada vez que se crea un nuevo símbolo, se crea un identificador único que no puede ser igual a ningún otro símbolo.

Datos Complejos:

1. **object:** el tipo de dato objeto en JavaScript se utiliza para representar un objeto, un arreglo o una función. Los objetos son colecciones de propiedades que se utilizan para representar entidades del mundo real, como una persona, una empresa o un coche. Los arreglos son listas de elementos, mientras que las funciones son bloques de código que realizan una tarea específica.

2. **function:** el tipo de dato función en JavaScript se utiliza para definir una función. Las funciones son bloques de código que se utilizan para realizar una tarea específica y pueden ser reutilizadas en diferentes partes de un programa. Las funciones también pueden tomar argumentos y devolver valores.

Es importante tener en cuenta que a los objetos y las funciones se los considera así porque pueden contener otros tipos de datos y proporcionar funcionalidad más compleja.

¿Qué son las variables en JavaScript?

En JavaScript, una variable es un contenedor que almacena un valor que puede ser utilizado o modificado más adelante en el programa. Para declarar una variable en JavaScript, se utilizan las palabras clave `var`, `let` o `const`, seguidas del nombre de la variable.

Por Ejemplo:

1. **var:** en versiones anteriores de JavaScript, se utilizaba la palabra clave `var` para declarar variables. Aunque aún se utiliza, se recomienda utilizar `let` y `const` en su lugar debido a que `var` puede llevar a errores de alcance. Las variables declaradas con `var` tienen un alcance global o de función, lo que significa que pueden ser accedidas desde cualquier parte del programa.

```
var nombre = "Juan";  
var edad = 25;
```

2. **let:** la palabra clave let se utiliza para declarar variables que tienen un alcance de bloque, lo que significa que solo pueden ser accedidas dentro del bloque en el que fueron declaradas. Por ejemplo, si se declaran dentro de una función, solo se pueden acceder dentro de esa función.

```
let nombre = "María";  
let edad = 30;
```

3. **const:** La palabra clave const se utiliza para declarar variables que no pueden ser modificadas después de ser declaradas. Es decir, su valor no puede ser reasignado. Las variables declaradas con const también tienen un alcance de bloque.

```
const PI = 3.14159;  
const nombre = "Pedro";
```

También es importante mencionar que las variables en JavaScript son de tipado dinámico, lo que significa que el tipo de dato de una variable puede cambiar durante la ejecución del programa.

Por Ejemplo:

```
let x = 10; // x es un número  
x = "Hola"; // x es ahora una cadena de texto
```

¿Cómo deben ser los nombres de las variables?

Al elegir un nombre para una variable, se deben seguir las siguientes reglas:

1. Los nombres de variables deben comenzar con una letra, un guión bajo (_) o un signo de dólar (\$).
2. Los nombres de variables pueden contener letras, números, guiones bajos (_) y signos de dólar (\$).
3. Los nombres de variables distinguen entre mayúsculas y minúsculas. Es decir, nombre y Nombre son dos variables diferentes.

Además, hay ciertas convenciones que se deben seguir para nombrar las variables correctamente:

1. Utilizar nombres de variables en minúsculas con palabras separadas por guiones bajos (_).
2. No utilizar palabras reservadas de JavaScript como nombres de variables. Las palabras reservadas son palabras clave que tienen un significado especial en el lenguaje, como if, for, while, true, false, entre otros.
3. Utilizar nombres de variables que sean descriptivos y reflejen su función en el programa. Por ejemplo, nombreCompleto es un nombre de variable más descriptivo que n o nc.
4. Evite nombres de variables demasiado largos o complicados. Los nombres de variables deben ser fáciles de leer y comprender.
5. Evite nombres de variables confusos o engañosos. Los nombres de variables deben representar con precisión su contenido para evitar confusiones.

En resumen, se deben utilizar nombres de variables descriptivos y legibles que reflejen su función en el programa, seguir las convenciones de nomenclatura comunes, evitar palabras reservadas y nombres confusos, y mantener los nombres de variables cortos y fáciles de leer.

¿Cuáles son las formas de inicializar las variables?

En JavaScript, se pueden inicializar las variables de varias formas, las formas más comunes son:

1. **Declaración y asignación en dos pasos:** Primero se declara la variable utilizando la palabra clave var, let o const, y luego se asigna un valor a la variable. Por ejemplo:

```
let nombre; // Declaración  
nombre = 'Juan'; // Asignación
```

2. **Declaración y asignación en un solo paso:** Se puede declarar y asignar un valor a la variable en un solo paso utilizando la palabra clave var, let o const, seguida del valor que se le quiere asignar. Por ejemplo:

```
let apellido = 'Pérez'; // Declaración y asignación
```

3. **Declaración múltiple:** Se pueden declarar varias variables al mismo tiempo utilizando la misma palabra clave `var`, `let` o `const`, separando cada variable con una coma y asignando un valor opcional a cada variable. Por ejemplo:

```
let edad, direccion, telefono;  
edad = 30;  
direccion = 'Calle 123';  
telefono = '555-1234';
```

4. **Asignación múltiple:** Se pueden asignar valores a varias variables al mismo tiempo utilizando el operador de asignación (`=`) y separando cada variable con una coma. Por ejemplo:

```
let nombre = 'Ana', apellido = 'González', edad = 25;
```

¿Qué es el ámbito de las variables en JavaScript?

El ámbito (o `scope`) de una variable se refiere al alcance o rango en el que dicha variable puede ser accedida. En otras palabras, el ámbito de una variable determina en qué partes del programa la variable es visible y puede ser utilizada.

JavaScript tiene tres tipos de ámbito o `scope`: global, local y de bloque.

1. **Ámbito global:** Las variables globales se declaran fuera de cualquier función o bloque de código y están disponibles en todo el programa. Una variable global puede ser accedida desde cualquier lugar del código, lo que puede ser conveniente, pero también puede ser peligroso si se utiliza de manera incorrecta. Por Ejemplo:

```
let nombre = 'Juan'; // variable global  
  
function saludar() {  
    console.log('Hola ' + nombre);  
}  
  
saludar(); // salida: Hola Juan
```

En el ejemplo, la variable `nombre` es una variable global que se declara fuera de la función `saludar()` y está disponible en todo el programa.

2. **Ámbito local:** Las variables que se definen dentro de una función o un bloque de código tienen un ámbito local, lo que significa que solo están disponibles dentro de la función o el bloque. Estas no pueden ser accesadas desde fuera de ella. Este ámbito se utiliza para limitar la visibilidad de una variable a la función en la que se encuentra. Por ejemplo:

```
function saludar() {  
  let nombre = 'Juan'; // variable local  
  console.log('Hola ' + nombre);  
}  
  
saludar(); // salida: Hola Juan  
  
console.log(nombre); // error: nombre is not defined
```

En este ejemplo, la variable nombre se declara dentro de la función saludar() y solo está disponible dentro de esa función. Si se intenta acceder a la variable fuera de la función, se producirá un error.

3. **Ámbito de Bloque:** Las variables declaradas con let o const dentro de un bloque de código (como un if, for o while) solo están disponibles dentro de ese bloque. Este ámbito se utiliza para limitar la visibilidad de una variable a una sección específica del código. Por ejemplo:

```
if (true) {  
  let nombre = 'Juan'; // variable local de bloque  
  console.log('Hola ' + nombre);  
}  
  
console.log(nombre); // error: nombre is not defined
```

En este ejemplo, la variable nombre se declara dentro del bloque if y solo está disponible dentro de ese bloque. Si se intenta acceder a la variable fuera del bloque, se producirá un error.

¿Qué son las Estructuras de control de flujo?

Las estructuras de control de flujo son un conjunto de instrucciones en un programa que controlan la ejecución de diferentes bloques de código. En JavaScript, existen tres tipos principales de estructuras de control de flujo: condicionales, bucles y saltos.

Estructuras de control de flujo condicionales

Las estructuras de control de flujo condicionales permiten al programa tomar decisiones basadas en ciertas condiciones. En JavaScript, las estructuras de control condicionales más comunes son:

1. **if/else:** La estructura de control if...else es utilizada para ejecutar un bloque de código si una condición es verdadera y otro bloque de código si la condición es falsa. La sintaxis es la siguiente:

```
if (condición) {  
    // código si la condición es verdadera  
} else {  
    // código si la condición es falsa  
}
```

Por Ejemplo:

```
let edad = 18;  
  
if (edad >= 18) {  
    console.log("Eres mayor de edad");  
} else {  
    console.log("Eres menor de edad");  
}
```

En este ejemplo, se verifica si el número es mayor o igual que 18 y se imprime un mensaje diferente en cada caso.

2. switch: La estructura de control switch se utiliza para seleccionar uno de varios bloques de código para ser ejecutado. La sintaxis es la siguiente:

```
switch (valor) {  
  case valor1:  
    // código si valor es igual a valor1  
    break;  
  case valor2:  
    // código si valor es igual a valor2  
    break;  
  default:  
    // código si valor no es igual a ninguno de los valores anteriores  
}
```

Por Ejemplo:

```
let dia = 3;  
  
switch (dia) {  
  case 1:  
    console.log("Lunes");  
    break;  
  case 2:  
    console.log("Martes");  
    break;  
  case 3:  
    console.log("Miércoles");  
    break;  
  default:  
    console.log("Día no válido");  
}
```

En este ejemplo, se verifica el día de la semana y se imprime un mensaje diferente en cada caso.

Estructuras de control de flujo de bucles

Las estructuras de control de flujo de bucles permiten ejecutar un bloque de código varias veces. En JavaScript, las estructuras de control de flujo de bucles más comunes son:

1. **for:** La estructura de control for se utiliza para repetir un bloque de código un número determinado de veces. La sintaxis es la siguiente:

```
for (inicialización; condición; incremento/decremento) {  
    // código a repetir  
}
```

Por Ejemplo:

```
for (let i = 0; i < 10; i++) {  
    console.log(i);  
}
```

En este ejemplo, se utiliza un bucle for para imprimir los números del 0 al 9.

2. **while:** La estructura de control while se utiliza para repetir un bloque de código mientras se cumpla una condición determinada. La sintaxis es la siguiente:

```
while (condición) {  
    // código a repetir  
}
```

Por Ejemplo:

```
let i = 0;  
  
while (i < 10) {  
    console.log(i);  
    i++;  
}
```

En este ejemplo, se utiliza un bucle while para imprimir los números del 0 al 9.

3. **do/while:** La estructura de control do/while es una variante de la estructura while. La diferencia radica en que, en este caso, el bloque de código se ejecuta al menos una vez antes de evaluar la condición de salida. La sintaxis es la siguiente:

```
do {  
    // código a repetir  
} while (condición);
```

Por Ejemplo:

```
let i = 0;  
  
do {  
    console.log(i);  
    i++;  
} while (i < 10);
```

En este ejemplo, se utiliza un bucle do/while para imprimir los números del 0 al 9.

Estructuras de control de flujo de saltos

Las estructuras de control de flujo de saltos permiten saltar a diferentes partes del código en función de ciertas condiciones. En JavaScript, las estructuras de control de flujo de saltos más comunes son:

1. **break:** La estructura de control break se utiliza para salir de un bucle o una estructura condicional. La sintaxis es la siguiente:

```
while (condición) {  
    // código a repetir  
    if (algunaCondicion) {  
        break;  
    }  
    // código que se ejecuta si algunaCondicion es falsa  
}
```

Por Ejemplo:

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) {  
    break;  
  }  
  console.log(i);  
}
```

En este ejemplo, la sintaxis de break se encuentra dentro de la declaración if y se ejecuta cuando la variable i es igual a 5. La ejecución de break interrumpe la iteración actual del bucle for y sale del bucle inmediatamente.

2. **continue:** La estructura de control continue se utiliza para saltar a la siguiente iteración de un bucle. La sintaxis es la siguiente:

```
while (condición) {  
  // código a repetir  
  if (algunaCondicion) {  
    continue;  
  }  
  // código que se ejecuta si algunaCondicion es falsa  
}
```

Por Ejemplo:

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) {  
    continue;  
  }  
  console.log(i);  
}
```

En este ejemplo, el código omite el número 5, no lo imprime en la consola, ya que se salta la iteración en la que i es igual a 5.

¿Qué es el operador ternario?

El operador ternario en JavaScript es una forma abreviada de escribir una estructura de control if-else en una sola línea de código. La sintaxis del operador ternario es la siguiente:

condición ? expresión verdadera : expresión falsa

La condición es una expresión que se evalúa como verdadera o falsa. Si la condición es verdadera, se devuelve la expresión verdadera, de lo contrario, se devuelve la expresión falsa.

Un ejemplo de cómo se puede usar el operador ternario en JavaScript para determinar si una persona es mayor de edad es el siguiente:

```
let edad = 18;  
let mensaje = (edad >= 18) ? "Eres mayor de edad" : "Eres menor de edad";  
console.log(mensaje);
```

En este ejemplo, la condición es `edad >= 18`. Si la edad es mayor o igual a 18, la expresión verdadera es "Eres mayor de edad". De lo contrario, la expresión falsa es "Eres menor de edad".

También se puede anidar el operador ternario para realizar múltiples comprobaciones. Aquí hay un ejemplo de cómo se puede usar el operador ternario para determinar si una persona es un adulto joven:

```
let edad = 18;  
let mensaje = (edad >= 18) ? "Eres mayor de edad" : "Eres menor de edad";  
console.log(mensaje);
```

En este ejemplo, se comprueba si la edad es mayor o igual a 18 y si es menor o igual a 25. Si la edad está dentro de ese rango, el mensaje será "Eres un adulto joven". De lo contrario, el mensaje será "Eres mayor de 25 años". Si la edad es menor de 18, el mensaje será "Eres menor de edad".

¿Qué es Interpolación de strings?

La interpolación de strings en JavaScript es una técnica que permite insertar variables y expresiones dentro de una cadena de texto. Es una forma más sencilla y legible de construir cadenas de texto dinámicamente, en comparación con la concatenación de strings.

La interpolación de strings se realiza utilizando el template literal, que es una cadena de texto que se encierra entre comillas invertidas (` ) en lugar de comillas simples o dobles. Dentro del template literal, se pueden incluir variables o expresiones dentro de llaves ( `{ }`) que serán evaluadas y reemplazadas por su valor correspondiente.

Un ejemplo de cómo se puede utilizar la interpolación de strings en JavaScript es el siguiente:

```
let nombre = "Juan";  
let edad = 25;  
console.log(`Hola, mi nombre es ${nombre} y tengo ${edad} años`);
```

En este ejemplo, se utiliza el template literal para construir una cadena de texto que incluye las variables `nombre` y `edad`. Dentro de la cadena de texto, se utilizan llaves para indicar que se desea incluir el valor de la variable correspondiente. El resultado de la ejecución de este código sería:

```
Hola, mi nombre es Juan y tengo 25 años
```

También se pueden incluir expresiones dentro de las llaves para realizar cálculos u operaciones antes de incluir el resultado dentro de la cadena de texto. Aquí hay un ejemplo de cómo se puede utilizar la interpolación de strings con una expresión:

```
let precio = 100;  
let descuento = 0.1;  
console.log(`El precio final es ${precio - (precio * descuento)} dólares`);
```

En este ejemplo, se utiliza la interpolación de strings para calcular el precio final de un producto después de aplicar un descuento del 10%. Dentro de las llaves se incluye la expresión `precio - (precio * descuento)` que se evalúa antes de incluir el resultado dentro de la cadena de texto. El resultado de la ejecución de este código sería:

El precio final es 90 dólares

En resumen, la interpolación de strings en JavaScript es una técnica útil para construir cadenas de texto dinámicamente y de forma legible utilizando el template literal y las llaves para incluir variables y expresiones dentro de la cadena de texto.

¿Para qué sirve el uso de la consola?

La consola en JavaScript es una herramienta que nos permite interactuar con nuestro código y depurar errores. Se trata de una interfaz de línea de comandos que nos permite ejecutar código JavaScript en tiempo real y ver los resultados en la misma ventana del navegador.

La consola se utiliza principalmente para:

1. **Depurar errores en el código:** podemos usar la consola para ver los errores que se producen en nuestro código y encontrar su origen.

Por Ejemplo:

```
function sumar(a, b) {  
  console.log("a:", a, "b:", b);  
  return a + b;  
}  
  
let resultado = sumar(2, "3");  
console.log(resultado);
```

Este código imprime los valores de las variables a y b en la consola antes de realizar la suma. Al ejecutar el código, podemos ver que el valor de b es "3", estamos intentando sumar un número y una cadena de texto, esto produce un resultado inesperado y puede causar errores en el programa. De esta manera nos permite encontrar el error y corregirlo.

2. **Probar código en tiempo real:** podemos utilizar la consola para probar pequeños fragmentos de código y ver los resultados de inmediato.

Por Ejemplo:

```
let x = 10;  
let y = 20;  
let resultado = x + y;  
console.log(resultado);
```

Este código suma las variables x e y, y asigna el resultado a la variable resultado. Al imprimir el valor de resultado en la consola, podemos ver que es 30. Podemos utilizar la consola para probar diferentes valores de x e y, y ver los resultados en tiempo real.

3. **Imprimir mensajes o valores de variables:** podemos usar la consola para imprimir mensajes o valores de variables en la ventana del navegador.

Por Ejemplo:

```
console.log("Hola, mundo!");
```

Este código imprime el mensaje "Hola, mundo!" en la consola.

-3. Depuración y Funciones

¿Qué es la Depuración?

La depuración (o debugging en inglés) en JavaScript es el proceso de identificar y corregir errores en el código de un programa. Es una parte importante del desarrollo de software ya que ayuda a garantizar que el código funcione correctamente y sin errores.

Existen varias herramientas y técnicas que se pueden utilizar para depurar el código en JavaScript. Algunas de las más comunes son:

1. **Consola de desarrolladores:** La mayoría de los navegadores web modernos tienen una consola de desarrolladores integrada que permite inspeccionar el código y ejecutar comandos en tiempo real. La consola es una herramienta útil para identificar errores en el código y para depurar problemas de lógica.

2. Puntos de interrupción: Los puntos de interrupción son puntos específicos en el código donde se detiene la ejecución para permitir la inspección del estado del programa. Los desarrolladores pueden utilizar los puntos de interrupción para inspeccionar el estado de las variables y para examinar el flujo de ejecución del programa.

3. Impresión de mensajes de depuración: Los mensajes de depuración son mensajes que se imprimen en la consola o en el código para ayudar a los desarrolladores a entender lo que está sucediendo en el programa. Los mensajes de depuración pueden ser útiles para rastrear errores y para entender el flujo de ejecución del programa.

Por Ejemplo:

Supongamos que tenemos un programa que debe calcular la suma de los primeros 10 números naturales, pero el resultado que obtenemos es incorrecto. Para depurar el código, podríamos agregar una sentencia de impresión en la consola para mostrar el valor de la suma en cada iteración del bucle, y así identificar el error.

```
let sum = 0;

for (let i = 1; i <= 10; i++) {
  sum += i; console.log("Valor actual de la suma: " + sum);
}
console.log("La suma de los primeros 10 números naturales es: " + sum);
```

Al ejecutar este código, se imprimirán los valores de la suma en cada iteración del bucle en la consola, lo que nos permitirá identificar si el valor de la suma está aumentando correctamente en cada iteración.

¿Qué son los Breakpoints?

Los breakpoints (puntos de interrupción en español) en JavaScript son una herramienta utilizada para detener la ejecución del código en un punto específico, permitiendo al desarrollador inspeccionar el estado del programa en ese momento y depurar el código.

Al colocar un breakpoint en una línea de código específica, la ejecución del programa se detendrá en esa línea y permitirá al desarrollador examinar el estado de las variables y objetos en ese momento. Esta herramienta es muy útil para entender el flujo de ejecución del programa y para identificar errores en el código.

Los breakpoints se pueden establecer en la mayoría de los navegadores modernos, utilizando la herramienta de depuración integrada. Los desarrolladores pueden colocar los breakpoints haciendo clic en el número de línea correspondiente en el panel de fuentes de la herramienta de depuración.

Una vez que se establece un breakpoint, el programa se detendrá en esa línea de código cuando se ejecute. A partir de ahí, el desarrollador puede inspeccionar el estado de las variables y objetos y utilizar otras herramientas de depuración para entender el flujo de ejecución y corregir errores en el código

Por Ejemplo:

Supongamos que tenemos el siguiente código que toma dos números como entrada y muestra el mayor de los dos:

```
function mostrarMayor(a, b) {  
  if (a > b) {  
    console.log(a);  
  } else {  
    console.log(b);  
  }  
}  
  
mostrarMayor(5, 3);
```

Si queremos depurar este código y ver qué sucede cuando se llama con ciertos valores, podemos establecer un breakpoint en la línea que contiene la llamada a la función mostrarMayor:

```
mostrarMayor(5, 3);
```

Para establecer un breakpoint en esta línea de código en la herramienta de depuración de Google Chrome, podemos hacer clic en el número de línea correspondiente en el panel de fuentes.

Una vez que se establece el breakpoint, podemos recargar la página y la ejecución del programa se detendrá en esa línea de código. A partir de ahí, podemos examinar los valores de a y b y utilizar otras herramientas de depuración para entender el flujo de ejecución y corregir cualquier error en el código.

En este ejemplo, podemos cambiar los valores de entrada para ver cómo el programa responde a diferentes situaciones. Podemos cambiar la llamada a

mostrarMayor(5, 3) a mostrarMayor(1, 10) y ver cómo el programa responde a estos nuevos valores. Con los breakpoints, podemos detener la ejecución en puntos específicos y examinar el estado del programa en ese momento, lo que nos ayuda a comprender cómo funciona nuestro código y a depurarlo de manera más efectiva.

¿Qué son las Funciones y los Argumentos?

En JavaScript, Las funciones son bloques de código reutilizables que se pueden llamar y ejecutar varias veces en diferentes partes del programa. Las funciones pueden tomar argumentos como entrada y pueden devolver un valor como salida. Las funciones son una parte fundamental de la programación en JavaScript y se utilizan ampliamente en la creación de aplicaciones web y en la manipulación de datos.

Los argumentos de una función en JavaScript son valores que se pasan a la función cuando se la llama. Las funciones pueden tener cero o más argumentos, y cada argumento puede ser de cualquier tipo de datos válido en JavaScript, incluyendo números, cadenas, objetos, arreglos y funciones. Los argumentos son una forma importante de hacer que las funciones sean más flexibles y adaptables a diferentes situaciones.

En JavaScript existen diferentes tipos de funciones y diferentes tipos de argumentos que se pueden utilizar. Como por ejemplo:

Tipos de funciones en JavaScript:

1. **Funciones declarativas:** se definen utilizando la palabra clave "function" y se pueden invocar en cualquier parte del código, incluso antes de ser declaradas.

Por Ejemplo:

```
function sumar(a, b) {  
  return a + b;  
}  
  
sumar(3, 5); // devuelve 8
```

2. **Funciones expresivas:** se definen utilizando una expresión y se asignan a una variable. Estas funciones no se pueden invocar antes de ser definidas.

Por Ejemplo:

```
const sumar = function(a, b) {  
  return a + b;  
};  
  
sumar(3, 5); // devuelve 8
```

3. **Funciones flechas:** son una forma abreviada de definir funciones expresivas y se utilizan para escribir código más conciso.

Por Ejemplo:

```
const sumar = (a, b) => a + b;  
  
sumar(3, 5); // devuelve 8
```

Tipos de Argumentos en JavaScript:

1. **Sin argumentos:** no reciben ningún valor como entrada. Es decir, no necesitan que se les pase ningún valor para poder ser ejecutadas y realizar su tarea.

Por Ejemplo:

```
function saludar() {  
  console.log("¡Hola! ¿Cómo estás?");  
}  
  
saludar(); // muestra por consola "¡Hola! ¿Cómo estás?"
```

En este ejemplo, la función "saludar" no recibe ningún argumento, pero realiza la tarea de mostrar un mensaje de saludo por consola cuando es invocada mediante la sentencia saludar();.

2. **Argumentos obligatorios:** son los argumentos que se deben pasar a una función para que esta pueda funcionar correctamente.

Por Ejemplo:

```
function sumar(a, b) {  
  return a + b;  
}  
  
sumar(3, 5); // devuelve 8
```

En este ejemplo, los argumentos "a" y "b" son obligatorios para que la función "sumar" pueda devolver un resultado.

3. **Argumentos por defecto:** son valores que se asignan a los argumentos de una función en caso de que no se pasen explícitamente.

Por Ejemplo:

```
function sumar = (a = 0, b = 0) {  
  return a + b;  
};  
  
sumar(); // devuelve 0
```

En este ejemplo, los argumentos "a" y "b" tienen un valor por defecto de cero, por lo que si no se pasan explícitamente, la función devolverá cero.

¿Qué son los objetos?

En JavaScript, los objetos son estructuras de datos que permiten almacenar información en forma de propiedades y métodos. Las propiedades son variables que almacenan valores y los métodos son funciones que pueden ser invocadas para realizar operaciones sobre los datos almacenados en el objeto.

Los objetos en JavaScript se definen mediante llaves { } y contienen una lista de pares clave-valor. La clave es un identificador y el valor puede ser cualquier tipo de datos, como números, cadenas, booleanos, matrices u otros objetos.

Por Ejemplo:

```
const persona = {  
  nombre: "Juan",  
  apellido: "Pérez",  
  edad: 30,  
  ciudad: "Buenos Aires",  
  saludar: function() {  
    console.log("Hola, soy " + persona.nombre + " " + persona.apellido);  
  }  
};
```

En este ejemplo, hemos creado un objeto llamado persona con cuatro propiedades (nombre, apellido, edad y ciudad) y un método llamado saludar, que imprime un mensaje de saludo en la consola.

Para acceder a las propiedades y métodos de un objeto, utilizamos la notación de punto (.) o la notación de corchetes ([]).

Por ejemplo, para acceder a la propiedad nombre del objeto persona, podemos hacer lo siguiente:

```
console.log(persona.nombre); // imprime "Juan"
```

También podemos invocar el método saludar de la siguiente manera:

```
persona.saludar(); // imprime "Hola, soy Juan Pérez"
```

Además, podemos agregar nuevas propiedades y métodos a un objeto existente o eliminarlos utilizando las mismas notaciones.

Por ejemplo, podemos agregar una nueva propiedad al objeto persona de la siguiente manera:

```
persona.telefono = "123456789";  
console.log(persona.telefono); // imprime "123456789"
```

En resumen, los objetos en JavaScript son estructuras de datos flexibles y poderosas que nos permiten almacenar y manipular información de manera eficiente y estructurada.

¿Qué son los arreglos?

En JavaScript, los arreglos son estructuras de datos que se utilizan para almacenar una colección de elementos, como números, cadenas de texto u objetos. Los arreglos son útiles cuando necesitas almacenar una lista de elementos que se relacionan entre sí y que quieres acceder de forma rápida y eficiente.

Para crear un arreglo en JavaScript, se utiliza la sintaxis de corchetes []. Dentro de los corchetes, se pueden agregar los elementos que conforman el arreglo, separados por comas.

Por Ejemplo:

```
let numeros = [1, 2, 3, 4, 5];  
let nombres = ["Juan", "Maria", "Pedro", "Ana"];  
let mixto = [1, "dos", true, {nombre: "Juan", edad: 25}];
```

Algunos Ejemplos son los siguientes:

Para acceder a un elemento específico dentro del arreglo, se utiliza la sintaxis de corchetes y se indica el índice del elemento que se quiere obtener. Los índices en JavaScript comienzan en cero, por lo que el primer elemento de un arreglo tiene un índice de 0.

Por Ejemplo:

```
let miArreglo = [1, 2, 3];
console.log(miArreglo[0]); // imprime 1
console.log(miArreglo[1]); // imprime 2
console.log(miArreglo[2]); // imprime 3
```

Es posible modificar un elemento dentro del arreglo simplemente asignándole un nuevo valor utilizando la sintaxis de corchetes.

Por Ejemplo:

```
let miArreglo = [1, 2, 3];
miArreglo[1] = 4;
console.log(miArreglo); // imprime [1, 4, 3]
```

Además de acceder a elementos individuales, es posible recorrer todo el arreglo utilizando un bucle for.

Por Ejemplo:

```
let miArreglo = [1, 2, 3];
for (let i = 0; i < miArreglo.length; i++) {
  console.log(miArreglo[i]);
}
```

El método length se utiliza para obtener el número de elementos que hay en el arreglo. En el ejemplo anterior, el bucle recorre el arreglo desde el índice 0 hasta el índice miArreglo.length - 1, que en este caso es 2.

Es posible crear un arreglo vacío y luego agregar elementos utilizando la función `push`.

Por Ejemplo:

```
let miArreglo = [];  
miArreglo.push(1);  
miArreglo.push(2);  
miArreglo.push(3);
```

Es posible eliminar el último elemento de un arreglo, se utiliza el método `pop()`.

Por Ejemplo:

```
let numeros = [1, 2, 3, 4];  
numeros.pop();  
console.log(numeros); // Imprime [1, 2, 3]
```

También existen otros métodos útiles para manipular arreglos en JavaScript, como `slice`, que permite obtener una porción del arreglo, o `sort`, que permite ordenar los elementos del arreglo.

Por Ejemplo:

```
let miArreglo = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3];  
let porcion = miArreglo.slice(2, 6);  
porcion.sort();  
console.log(porcion); // imprime [1, 4, 5, 9]  
console.log(miArreglo); // imprime [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
```

En este ejemplo, se obtiene una porción del arreglo que contiene los elementos desde el índice 2 hasta el índice 5 ([4, 1, 5, 9]). Luego, se utiliza el método `sort` para ordenar los elementos de la porción ([1, 4, 5, 9]). Cabe destacar que el método `sort` modifica el arreglo original, por lo que después de ordenar la porción, el arreglo completo se mantiene sin cambios.

Hay muchos otros métodos y operaciones que se pueden realizar con arreglos en JavaScript.

¿Qué es la desestructuración de objetos?

La desestructuración de objetos en JavaScript es una característica que permite extraer valores de un objeto y asignarlos a variables individuales, en una sola línea de código. Esto es especialmente útil cuando necesitamos acceder a varias propiedades de un objeto en diferentes partes del código.

La sintaxis de la desestructuración de objetos es la siguiente:

```
let {propiedad1, propiedad2, ..., propiedadN} = objeto;
```

En esta sintaxis, se define un objeto y se asignan los valores de sus propiedades a las variables propiedad1, propiedad2, ..., propiedadN en el orden en que aparecen en el objeto

Algunos Ejemplos son los siguientes:

Se puede utilizar la desestructuración de objetos para asignar los valores de las propiedades de un objeto a variables individuales.

Por Ejemplo:

```
const miObjeto = {  
  nombre: "Juan",  
  apellido: "Pérez",  
  edad: 30,  
};  
  
const { nombre, apellido, edad } = miObjeto;  
  
console.log(nombre); // muestra "Juan"  
console.log(apellido); // muestra "Pérez"  
console.log(edad); // muestra 30
```

En este ejemplo, primero creamos un objeto llamado miObjeto con tres propiedades: nombre, apellido y edad. Luego, utilizamos la sintaxis de desestructuración para extraer cada una de las propiedades del objeto y asignarlas a variables individuales nombre, apellido y edad. Esto se hace mediante la sintaxis de llaves en el lado izquierdo de la asignación.

Es posible asignar valores predeterminados a las variables de desestructuración, en caso de que la propiedad correspondiente no exista en el objeto:

Por Ejemplo:

```
const miObjeto = {  
  nombre: "Juan",  
  apellido: "Pérez",  
  edad: 30,  
};  
  
const { nombre, apellido, edad } = miObjeto;  
  
console.log(nombre); // muestra "Juan"  
console.log(apellido); // muestra "Pérez"  
console.log(edad); // muestra 30
```

En este ejemplo, miObjeto solo tiene dos propiedades, por lo que al desestructurar el objeto y asignarlo a nombre, apellido y edad, edad recibirá el valor predeterminado de 30.

Además, también es posible utilizar alias para las variables de desestructuración, en caso de que se quiera dar un nombre diferente a la variable.

Por Ejemplo:

```
const miObjeto = {  
  nombre: "Juan",  
  apellido: "Pérez",  
};  
  
const { nombre: primerNombre, apellido: segundoNombre } = miObjeto;  
  
console.log(primerNombre); // muestra "Juan"  
console.log(segundoNombre); // muestra "Pérez"
```

En este ejemplo, utilizamos la sintaxis de alias para asignar las propiedades nombre y apellido del objeto a las variables primerNombre y segundoNombre, respectivamente.

En resumen, la desestructuración de objetos en JavaScript es una técnica útil para extraer valores de un objeto y asignarlos a variables individuales, de manera rápida y conveniente. Además, permite asignar valores predeterminados y utilizar alias para las variables de desestructuración.

¿Qué es la desestructuración de arreglos?

La desestructuración de arreglos en JavaScript es una característica que permite extraer valores de un arreglo y asignarlos a variables individuales, en una sola línea de código.

La sintaxis de la desestructuración de arreglos es la siguiente:

```
let [var1, var2, ..., varN] = arreglo;
```

En esta sintaxis, se define un arreglo y se asignan los valores del arreglo a las variables var1, var2, ..., varN en el orden en que aparecen en el arreglo.

Algunos Ejemplos son los siguientes:

Se puede utilizar la desestructuración de arreglos para asignar los valores de un arreglo a variables individuales.

Por Ejemplo:

```
let numeros = [1, 2, 3, 4, 5];  
let [a, b, c, d, e] = numeros;  
  
console.log(a); // 1  
console.log(b); // 2  
console.log(c); // 3  
console.log(d); // 4  
console.log(e); // 5
```

En este ejemplo, el arreglo numeros se asigna a las variables a, b, c, d y e mediante la desestructuración. Cada elemento del arreglo se asigna a una variable individual, lo que nos permite acceder a los valores de manera más fácil y legible.

Es posible omitir algunos elementos del arreglo utilizando una sintaxis similar a la desestructuración de objetos.

Por Ejemplo:

```
let numeros = [1, 2, 3, 4, 5];  
let [a, , c, , e] = numeros;  
  
console.log(a); // 1  
console.log(c); // 3  
console.log(e); // 5
```

En este ejemplo, los elementos en las posiciones 1, 3 y 5 del arreglo se omiten utilizando la sintaxis de "ignorar" la variable. Los valores de los elementos se saltan y no se asignan a ninguna variable.

Además, también es posible asignar valores predeterminados a las variables de desestructuración, en caso de que el valor correspondiente no exista en el arreglo,

Por Ejemplo:

```
const miArreglo = [1];  
const [a, b = 2] = miArreglo;  
  
console.log(a); // muestra 1  
console.log(b); // muestra 2 (valor predeterminado)
```

En este ejemplo, miArreglo solo tiene un elemento, por lo que al desestructurar el arreglo y asignarlo a a y b, b recibirá el valor predeterminado de 2.

En resumen, la desestructuración de arreglos en JavaScript es una técnica útil para extraer valores de un arreglo y asignarlos a variables individuales, de manera rápida y conveniente. Además, permite omitir elementos del arreglo y asignar valores predeterminados a las variables de desestructuración.

¿Qué es la Modularización (import y export)?

La modularización en JavaScript se refiere a la capacidad de dividir un programa en módulos más pequeños y manejables, cada uno de los cuales tiene una función específica. Esto ayuda a simplificar el código, hacerlo más mantenible y reutilizable, y reduce la posibilidad de errores y conflictos.

En JavaScript, la modularización se logra a través del uso de las palabras clave **"import"** y **"export"**.

"Export" se utiliza para hacer que una función, variable o clase esté disponible para otros módulos para su uso, mientras que **"Import"** se utiliza para acceder a las funciones, variables o clases que se han exportado desde otro módulo.

Por ejemplo:

si tenemos un archivo llamado "math.js" que contiene una función llamada "sumar" que toma dos números y devuelve su suma, podemos exportar esta función para su uso en otros archivos usando la siguiente sintaxis:

```
export function sumar(num1, num2) {  
  return num1 + num2;  
}
```

Luego, en otro archivo donde queremos usar la función "sumar", podemos importarla de la siguiente manera:

```
import { sumar } from './math.js';
```

Ahora podemos usar la función "sumar" en este archivo sin tener que volver a escribir su código. La modularización en JavaScript es una técnica útil y ampliamente utilizada en el desarrollo de aplicaciones web modernas.

-4. Manipulación del DOM y Asincronía

¿Qué es la Manipulación del DOM?

La Manipulación del DOM (Document Object Model) se refiere al proceso de modificar los elementos y contenido de una página web en tiempo real, utilizando JavaScript u otros lenguajes de programación. El DOM es una representación en memoria de la estructura de la página web, que permite a los desarrolladores

acceder y modificar los elementos HTML, CSS y JavaScript de la página.

La manipulación del DOM es una técnica fundamental en el desarrollo web moderno, ya que permite crear páginas dinámicas y reactivas que responden a las acciones del usuario y a los cambios en el estado de la aplicación.

Algunos ejemplos comunes de manipulación del DOM son:

1. Cambiar el contenido de un elemento HTML:

```
<p id="miParrafo">Este es un párrafo.</p>
```

```
const p = document.getElementById("miParrafo");  
p.innerHTML = "Este es un nuevo texto para el párrafo.";
```

En este ejemplo, se selecciona el elemento p con el ID = miParrafo y se cambia su contenido utilizando la propiedad innerHTML.

2. Modificar los estilos CSS de un elemento:

```
<div id="miDiv">Este es un div.</div>
```

```
const div = document.getElementById("miDiv");  
div.style.backgroundColor = "red";
```

En este ejemplo, se selecciona el elemento div con el ID = miDiv y se cambia su estilo de fondo utilizando la propiedad style.backgroundColor.

3. Agregar y eliminar elementos HTML dinámicamente:

```
<ul id="miLista">  
  <li>Elemento 1</li>  
  <li>Elemento 2</li>  
</ul>
```

```
const lista = document.getElementById("miLista");
const nuevoElemento = document.createElement("li");
nuevoElemento.textContent = "Nuevo elemento";
lista.appendChild(nuevoElemento);

const elementoEliminado = lista.removeChild(lista.firstElementChild);
```

En este ejemplo, se selecciona el elemento ul con el ID = miLista y se agrega un nuevo elemento li con el contenido "Elemento 3" utilizando el método createElement y appendChild. Luego, se elimina el primer elemento de la lista utilizando el método removeChild.

4. Agregar un evento cuando se haga un click:

```
<button id="miBoton">Clickea acá</button>
```

```
const miBoton = document.getElementById('miBoton');

miBoton.addEventListener('click', () => {
  miBoton.textContent = '¡Clickea de nuevo!';
});
```

Este código selecciona el botón utilizando document.getElementById() y luego agrega un evento de click usando el método addEventListener(). Primero le pasamos el tipo de evento que queremos "escuchar" del botón, en este caso será cada vez que se haga click, por ende ponemos "click" como primer parámetro. Como segundo parámetro, le pasamos una función flecha que ejecutará un bloque de código con la modificación o acción que deseemos, siguiendo con el ejemplo, hará el cambio del texto en el botón utilizando la propiedad .textContent y asignándole un nuevo valor.

La manipulación del DOM es una herramienta poderosa y versátil para crear páginas web interactivas y dinámicas. Sin embargo, es importante tener en cuenta que un uso excesivo o incorrecto de esta técnica puede afectar negativamente el rendimiento y la accesibilidad de la página web.

¿Qué son los Callbacks?

En JavaScript, un Callback es una función que se pasa como argumento a otra función y que se ejecuta después de que la función original haya completado su tarea. Los Callbacks son una técnica comúnmente utilizada en operaciones asíncronas, como la lectura de archivos o el envío de solicitudes HTTP, donde se necesita esperar a que la operación se complete antes de realizar la siguiente acción.

Los Callbacks son importantes en JavaScript porque permiten que el código asíncrono se comporte de manera sincrónica. En lugar de bloquear la ejecución del código hasta que se complete una tarea, se puede seguir ejecutando el código mientras se espera que se complete la tarea, y cuando la tarea se complete, se ejecutará el Callback correspondiente para continuar la ejecución del código.

Un Ejemplo de un Callback en JavaScript es el siguiente:

```
function mostrarMensaje(mensaje, callback) {  
  console.log(mensaje);  
  callback();  
}  
  
function mostrarMensajeDeConfirmacion() {  
  console.log("Mensaje mostrado");  
}  
  
mostrarMensaje("Este es un mensaje", mostrarMensajeDeConfirmacion);
```

En este ejemplo, la función `mostrarMensaje` toma dos argumentos: un mensaje y un Callback. La función muestra el mensaje en la consola y luego llama al Callback. La función `mostrarMensajeDeConfirmacion` simplemente muestra otro mensaje en la consola.

La función `mostrarMensaje` se llama con el mensaje "Este es un mensaje" y la función `mostrarMensajeDeConfirmacion` como Callback. Cuando se llama a la función `mostrarMensaje`, se muestra el mensaje "Este es un mensaje" y luego se llama a la función `mostrarMensajeDeConfirmacion`, que muestra el mensaje "Mensaje mostrado".

Los Callbacks también se utilizan a menudo en la programación asíncrona, por ejemplo, para manejar la respuesta de una solicitud HTTP.

Un ejemplo de cómo se puede utilizar un Callback para manejar la respuesta de una solicitud HTTP utilizando la biblioteca Axios en JavaScript es el siguiente:

```
const axios = require('axios');

axios.get('https://jsonplaceholder.typicode.com/todos/1')
  .then(function (response) {
    console.log(response.data);
  })
  .catch(function (error) {
    console.log(error);
  });
```

En este ejemplo, la función `axios.get` realiza una solicitud HTTP GET a una API en línea y devuelve una Promesa que se puede manejar con los métodos `then` y `catch`. Cuando se llama al método `then`, se ejecuta la función de Callback y se muestra el resultado de la solicitud en la consola. Si se produce un error, se llama al método `catch` y se muestra el error en la consola.

Los Callbacks son una técnica importante en JavaScript para trabajar con operaciones asíncronas. Sin embargo, los Callbacks anidados pueden hacer que el código sea difícil de leer y mantener. En tales casos, las Promesas y las `async/await` pueden ser mejores opciones para manejar operaciones asíncronas.

¿Qué son las Promesas?

Las Promesas en JavaScript son un mecanismo para trabajar con operaciones asíncronas de manera más clara y legible que utilizando callbacks anidados. Las promesas representan un valor que puede estar disponible ahora, en el futuro o nunca. Se utilizan en situaciones en las que se espera que una operación tome algún tiempo para completarse, como la lectura de un archivo, la consulta a una base de datos o el envío de una solicitud a un servidor.

Una promesa es un objeto que tiene tres estados posibles: `pending` (pendiente), `fulfilled` (cumplida) y `rejected` (rechazada). Cuando una promesa está pendiente, significa que la operación aún no se ha completado. Cuando una promesa se cumple, significa que la operación se ha completado con éxito y devuelve un valor. Cuando una promesa es rechazada, significa que la operación ha fallado y devuelve un error.

Para crear una promesa en JavaScript, se utiliza el constructor Promise. El constructor toma una función que recibe dos parámetros, resolve y reject. resolve se utiliza para cumplir la promesa y devolver el valor esperado, mientras que reject se utiliza para rechazar la promesa y devolver un error.

Un Ejemplo de cómo crear una promesa y cumplirla es el siguiente:

```
const miPromesa = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    resolve("Operación exitosa");  
  }, 2000);  
});  
  
miPromesa.then((resultado) => {  
  console.log(resultado); // "Operación exitosa"  
});
```

En este ejemplo, se crea una nueva promesa que espera dos segundos antes de cumplirse con éxito utilizando la función setTimeout. Luego, se utiliza el método then para acceder al resultado de la promesa una vez que se haya cumplido.

Un Ejemplo de cómo crear una promesa y rechazarla es el siguiente:

```
const miPromesa = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    reject("Operación fallida");  
  }, 2000);  
});  
  
miPromesa.catch((error) => {  
  console.error(error); // "Operación fallida"  
});
```

En este ejemplo, se crea una nueva promesa que espera dos segundos antes de rechazarse utilizando la función setTimeout. Luego, se utiliza el método catch para manejar el error y acceder al mensaje de error.

Las Promesas en JavaScript son una técnica poderosa para trabajar con operaciones asíncronas de manera clara y legible. La sintaxis de las promesas facilita la gestión de errores y el control del flujo de datos. Sin embargo, es importante tener en cuenta que las promesas pueden no ser adecuadas para todas las situaciones, especialmente cuando se trata de operaciones más complejas o que

requieren un mayor nivel de control.

Async/Await

¿Qué es sincrónico y asincrónico?

"sincrónico" se refiere a un comportamiento en el que una tarea se completa antes de pasar a la siguiente tarea. En otras palabras, una tarea no comienza hasta que la tarea anterior se haya completado. Esto se llama "bloqueo" o "espera" en el código, ya que el flujo de la aplicación se detiene hasta que la tarea actual se completa.

"asincrónico" se refiere a un comportamiento en el que una tarea se inicia pero no se completa de inmediato, permitiendo que el flujo de la aplicación continúe ejecutándose mientras se espera la finalización de la tarea. En otras palabras, una tarea no bloquea la ejecución de la tarea siguiente, y la ejecución del programa puede continuar mientras se espera a que la tarea actual se complete.

¿Cómo se realizan las tareas asíncronas?

Esto se logra a través de mecanismos como las promesas, los callbacks, las funciones asíncronas y el bucle de eventos. Al utilizar estas técnicas, se puede escribir código que maneje tareas que pueden tomar tiempo en completarse, como hacer una petición HTTP o acceder a una base de datos, sin bloquear la ejecución del resto del código.

¿Qué es Async/Await?

Async/Await es una técnica en JavaScript que se utiliza para trabajar con operaciones asincrónicas de manera más legible y fácil de entender que los Callbacks anidados o las Promesas encadenadas. Async/Await es una combinación de las palabras clave `async` y `await`, que se utilizan juntas para trabajar con funciones asincrónicas.

Las funciones marcadas con la palabra clave `async` devuelven una Promesa y se pueden utilizar con la palabra clave `await` dentro de otras funciones marcadas con la palabra clave `async`. La palabra clave `await` detiene la ejecución de la función hasta que se resuelve la Promesa, lo que permite que el código se comporte de manera sincrónica.

Un Ejemplo de cómo se puede utilizar Async/Await en JavaScript para trabajar con operaciones asincrónicas es el siguiente:

```
async function obtenerDatos() {  
  try {  
    const respuesta = await fetch('https://jsonplaceholder.typicode.com/todos/1');  
    const datos = await respuesta.json();  
    console.log(datos);  
  } catch (error) {  
    console.log(error);  
  }  
}  
  
obtenerDatos();
```

En este ejemplo, se define una función `obtenerDatos` con la palabra clave `async`. Dentro de la función, se utiliza la palabra clave `await` para detener la ejecución hasta que se resuelve la Promesa devuelta por la función `fetch`, que realiza una solicitud HTTP GET a una API en línea. Una vez que se resuelve la Promesa, se llama al método `json` para convertir la respuesta en un objeto JSON. La respuesta JSON se guarda en la variable `datos` y se muestra en la consola.

Si se produce un error durante la solicitud HTTP, se maneja utilizando el bloque `try/catch`, y el error se muestra en la consola.

Async/Await también se puede utilizar con funciones que devuelven Promesas, como la biblioteca Axios.

Un ejemplo de cómo se puede utilizar `Async/Await` con `Axios` en JavaScript es el siguiente:

```
const axios = require('axios');  
  
async function obtenerDatos() {  
  try {  
    const respuesta = await axios.get('https://jsonplaceholder.typicode.com/todos/1');  
    console.log(respuesta.data);  
  } catch (error) {  
    console.log(error);  
  }  
}  
  
obtenerDatos();
```

En este ejemplo, se utiliza la biblioteca `Axios` para realizar una solicitud HTTP GET a una API en línea. La función `axios.get` devuelve una Promesa, que se puede manejar

con la palabra clave `await` dentro de la función `obtenerDatos`. Una vez que se resuelve la Promesa, se muestra la respuesta en la consola.

En resumen, `Async/Await` es una técnica en JavaScript que se utiliza para trabajar con operaciones asíncronas de manera más legible y fácil de entender que los `Callbacks` anidados o las Promesas encadenadas. Las funciones marcadas con la palabra clave `async` devuelven una Promesa y se pueden utilizar con la palabra clave `await` dentro de otras funciones marcadas con la palabra clave `async`. La palabra clave `await` detiene la ejecución de la función hasta que se resuelve la Promesa, lo que permite que el código se comporte de manera sincrónica.

Consumo de API's con Fetch

¿Qué es una API?

Una API (Application Programming Interface) es un conjunto de reglas, protocolos y herramientas que se utilizan para interactuar con una aplicación o servicio web. Básicamente, una API proporciona una interfaz para que otras aplicaciones puedan interactuar con un servicio y obtener información de él.

¿Para qué se utilizan las API's?

Las API se utilizan para comunicar aplicaciones o servicios entre sí e intercambiar información y realizar acciones. Una API proporciona una interfaz estandarizada y bien definida que permite a los desarrolladores interactuar con una aplicación o servicio de manera programática, es decir, a través de código. Las API pueden tener diferentes niveles de complejidad y pueden estar diseñadas para una amplia variedad de usos, desde la integración de servicios en una aplicación hasta la automatización de tareas en sistemas empresariales. En resumen, las API son una herramienta fundamental para la integración de sistemas y servicios en la era digital.

¿Cómo se consumen las API's?

El consumo de una API con el método `fetch` en JavaScript es una forma de hacer una solicitud HTTP para obtener información de un servicio web. `Fetch` es un método nativo de JavaScript que proporciona una forma fácil de hacer solicitudes HTTP asíncronas. Se utiliza para hacer solicitudes HTTP y obtener respuestas del servidor. La respuesta se puede convertir en diferentes formatos, como JSON, texto, `ArrayBuffer` o `Blob`.

Un ejemplo de cómo consumir una API utilizando `fetch` en JavaScript es el siguiente:

```
fetch('https://api.example.com/data')  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.error(error));
```

En este ejemplo, se utiliza el método `fetch` para hacer una solicitud HTTP a la URL **`https://api.example.com/data`**. La función `fetch` devuelve una promesa que se resuelve con la respuesta de la solicitud.

Luego, se utiliza el método `json()` para convertir la respuesta en un objeto JSON. La función `json()` también devuelve una promesa que se resuelve con el objeto JSON.

Finalmente, se utiliza el método `then()` para imprimir los datos obtenidos a través de la solicitud HTTP en la consola. También se utiliza el método `catch()` para manejar cualquier error que pueda ocurrir durante la solicitud HTTP.

Es importante destacar que el consumo de una API con `fetch` en JavaScript puede variar dependiendo de los detalles específicos de la API que se esté consumiendo, como el tipo de datos que devuelve y los parámetros de la solicitud que se requieren.